

```
1 from tensorflow.keras.datasets.mnist import load_data
```

```
1 (X_train, y_train), (X_test, y_test) = load_data()
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-dataset/11493376/11490434> [=====] - 0s 0us/step
 11501568/11490434 [=====] - 0s 0us/step

```
1 print(X_train.shape, y_train.shape, X_test.shape, y_test.shape)
```

```
(60000, 28, 28) (60000,) (10000, 28, 28) (10000,)
```

```
1 import numpy as np
```

```
2 from matplotlib import pyplot as plt
```

```
3
```

```
4 i = np.random.randint(60000)
```

```
5
```

```
6 img = X_train[i]
```

```
7 print(i)
```

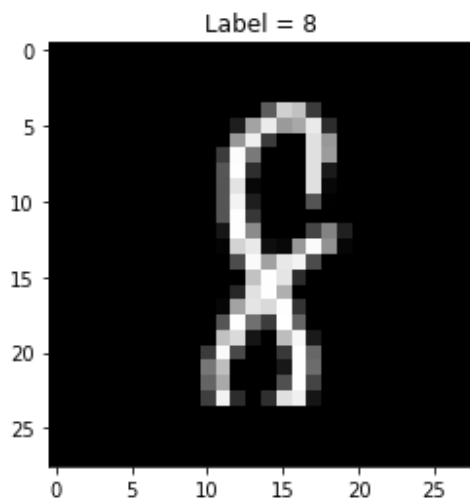
```
8
```

```
9 plt.imshow(img, cmap='gray')
```

```
10 plt.title(f"Label = {y_train[i]}")
```

```
28648
```

```
Text(0.5, 1.0, 'Label = 8')
```



```
1 y_train_onehot = np.zeros((X_train.shape[0], 10))
```

```
2 for i in range(X_train.shape[0]):
```

```
3     y_train_onehot[i, y_train[i]] = 1.0
```

```
4
```

```
5 y_test_onehot = np.zeros((X_test.shape[0], 10))
```

```
6 for i in range(X_test.shape[0]):
```

```
7     y_test_onehot[i, y_test[i]] = 1.0
```

```
8
```

```
9 print(y_train_onehot.shape, y_test_onehot.shape)
```

```
10 print(y_train_onehot[2])
```

```
11 print(y_train[2])
```

```
(60000, 10) (10000, 10)
[0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
4
```

```
1 X_train_norm = np.array([x.ravel() / 255. for x in X_train])
2 X_test_norm = np.array([x.ravel() / 255. for x in X_test])
3
4 print(X_train_norm.shape, X_test_norm.shape)
5
```

```
(60000, 784) (10000, 784)
```

```
1 g = lambda z: 1 / (1 + np.exp(-z))
2
3 class SigmoidLayer:
4     def __init__(self, n_inp, n_out, bias=True):
5         if bias: n_inp += 1
6         w = np.random.randn(n_inp, n_out)
7         self.w = w
8         self.bias = bias
9         self.is_forward = False
10
11     def forward(self, X):
12         if self.bias:
13             X = np.hstack((np.ones((X.shape[0], 1)), X))
14
15         z = np.dot(X, self.w)
16         a = g(z)
17
18         self.is_forward = True
19         self.x = X
20         self.a = a
21         return a
22
23     def back_propagation(self, delta):
24         a = self.a
25         dw = np.dot(self.x.T, delta * a * (1.0 - a))
26
27         if self.bias:
28             da = np.dot(delta * a * (1.0 - a), self.w.T)[: , 1:]
29         else:
30             da = np.dot(delta * a * (1.0 - a), self.w.T)
31
32         return dw, da
```

```
1 class SoftmaxLayer:
2     def __init__(self, n_inp, n_out, bias=True):
3         if bias: n_inp += 1
4         w = np.random.randn(n_inp, n_out)
5         self.w = w
6         self.bias = bias
7         self.is_forward = False
8
9     def forward(self, X):
```

```

10     if self.bias:
11         X = np.hstack((np.ones((X.shape[0], 1)), X))
12
13     z = np.dot(X, self.w)
14     e = np.exp(z)
15     s = np.sum(e, axis=1, keepdims=True)
16
17     self.is_forward = True
18     self.x = X
19
20     return e / s
21
22 def back_propagation(self, delta):
23     dw = np.dot(self.x.T, delta)
24     if self.bias:
25         da = np.dot(delta, self.w.T)[: , 1:]
26     else:
27         da = np.dot(delta, self.w.T)
28
29     return dw, da
30
31 def loss(y_true, y_pred):
32     return -np.sum(y_true * np.log(y_pred)) / y_true.shape[0]

1 # settings
2 NDIMS = 784
3 L1 = 64
4 L2 = 64
5 EPOCHS = 500
6 LR = 0.0001
7
8 layers = [SigmoidLayer(NDIMS, L1, bias=True),
9           SigmoidLayer(L1, L2),
10          SoftmaxLayer(L2, 10)]

1 def predict(X, model):
2     for l in model:
3         X = l.forward(X)
4     return X
5
6 def train_step(X, y, model, lr=0.01):
7     y_pred = predict(X, model)
8     delta = y_pred - y
9
10    for l in reversed(model):
11        dw, delta = l.back_propagation(delta)
12        l.w -= lr * dw
13
14 def momentum_train_step(X, y, model, lr=0.01, gamma=0.1):
15     y_pred = predict(X, model)
16     delta = y_pred - y
17
18    for l in reversed(model):
19        dw, delta = l.back_propagation(delta)
20        l.w -= (lr * dw + gamma * l.w)

```

```
1 y_pred = predict(X_train_norm, model=layers)
2 print(f"Initial loss: {loss(y_train_onehot, y_pred)}")
3
4 history_loss = []
5
6 for i in range(20):
7     train_step(X_train_norm, y_train_onehot, model=layers, lr=0.002)
8     y_pred = predict(X_train_norm, model=layers)
9
10    loss_ = loss(y_train_onehot, y_pred)
11    print(f"loss: {loss_}")
12
13    history_loss += [loss_]
```

Initial loss: 0.45418744409286754

loss: 0.451574650232667
loss: 0.44930502282921386
loss: 0.4469649770785338
loss: 0.4448612490651665
loss: 0.4427425659020044
loss: 0.44078258353689287
loss: 0.43884300554702815
loss: 0.43700941364902385
loss: 0.4352115118410097
loss: 0.4334896702938555
loss: 0.43179859675139187
loss: 0.4301747000487649
loss: 0.42855757394418753
loss: 0.42701717418325214
loss: 0.42544370719920777
loss: 0.4239708820485302
loss: 0.42241477723561105
loss: 0.42099212655811025
loss: 0.4194327046818915
loss: 0.4180421972279035
loss: 0.4164657008782065
loss: 0.4150901703546004
loss: 0.4134903157471279
loss: 0.4121152085227153
loss: 0.41049278490106256
loss: 0.40910767861226205
loss: 0.40746926677891865
loss: 0.4060687547001899
loss: 0.404424859273012
loss: 0.4030086011007672
loss: 0.40137159793258054
loss: 0.39994358697256566
loss: 0.3983258618941502
loss: 0.39689315531104047
loss: 0.3953056891256218
loss: 0.39387693380619865
loss: 0.39232843413199736
loss: 0.3909124937679496
loss: 0.38940904300878637
loss: 0.3880139318690146
loss: 0.3865590405132537
loss: 0.3851912485538269
loss: 0.383786174444966
loss: 0.38245036798900334
loss: 0.38109457062730906

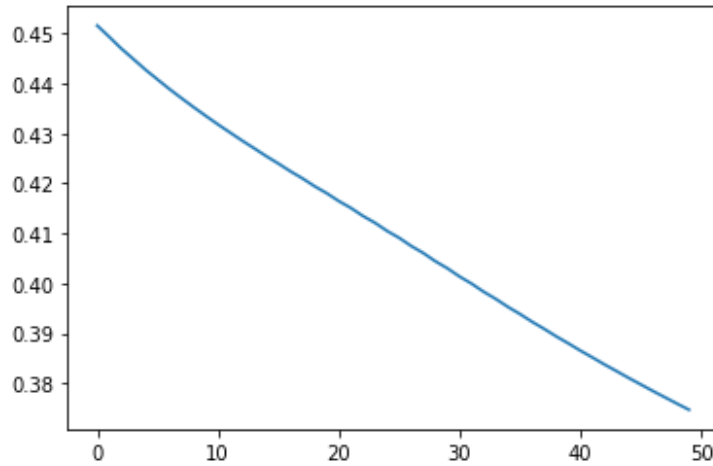
```

loss: 0.37979359195099927
loss: 0.37848521927023543
loss: 0.3772202864632293
loss: 0.3759566261218495
loss: 0.37472764013600385

```

```
1 plt.plot(history_loss)
```

```
[<matplotlib.lines.Line2D at 0x7f1a693a1690>]
```



```
1 y_pred = predict(X_test_norm, model=layers)
```

```

1 i = np.random.randint(10000)
2 print(i)
3 print(y_pred[i])
4 print(np.argmax(y_pred[i]))
5 print(y_test_onehot[i])
6 print(y_test[i])
7
8 plt.imshow(X_test[i], cmap='gray')

```

```

9331
[8.15281484e-10 5.17533848e-10 2.04760655e-14 2.13771250e-07
 1.96887352e-10 9.99992497e-01 2.14785174e-11 2.91058335e-14
 1.08150982e-07 7.17984248e-06]

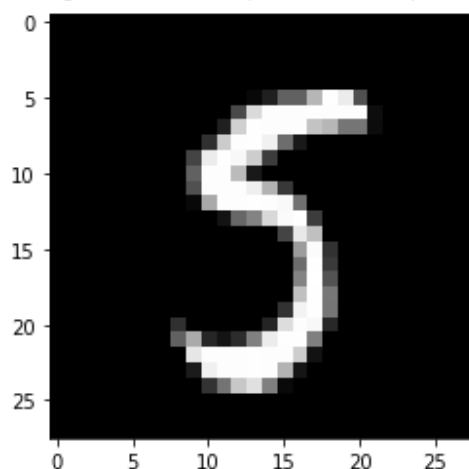
```

```
5
```

```
[0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
```

```
5
```

```
<matplotlib.image.AxesImage at 0x7f1a691cc190>
```



```
1 y_pred = np.argmax(y_pred, axis=1)
```

```
1 from sklearn.metrics import classification_report
2 print(classification_report(y_test, y_pred))
3
```

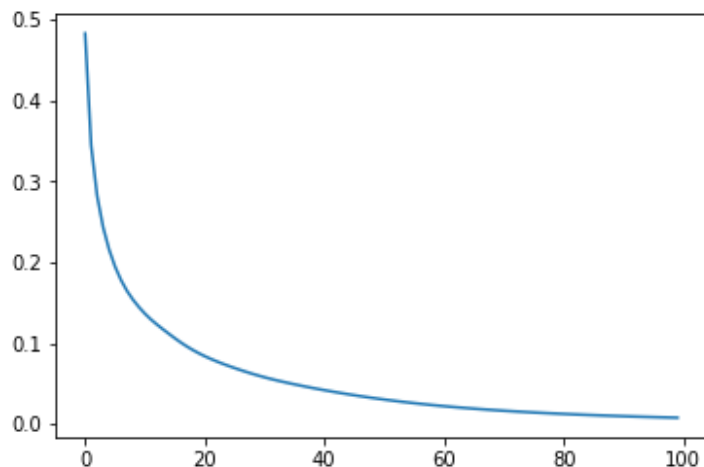
	precision	recall	f1-score	support
0	0.97	0.98	0.97	980
1	0.98	0.98	0.98	1135
2	0.94	0.96	0.95	1032
3	0.94	0.94	0.94	1010
4	0.95	0.95	0.95	982
5	0.93	0.94	0.93	892
6	0.97	0.95	0.96	958
7	0.95	0.95	0.95	1028
8	0.95	0.92	0.93	974
9	0.95	0.92	0.93	1009
accuracy			0.95	10000
macro avg	0.95	0.95	0.95	10000
weighted avg	0.95	0.95	0.95	10000

```
1 def batch_generator(X, y, batch_size=32):
2     idx = np.arange(X.shape[0])
3     np.random.shuffle(idx)
4
5     n_batch = len(idx) // batch_size
6     for i in range(n_batch):
7         i_start = i * batch_size
8         i_stop = min((i + 1) * batch_size, len(idx))
9         yield X[i_start:i_stop, :], y[i_start:i_stop, :]
10
11 # Creating new models
12 NDIMS = 784
13 L1 = 64
14 L2 = 64
15 EPOCHS = 500
16 LR = 0.0001
17
18 layers = [SigmoidLayer(NDIMS, L1, bias=True),
19           SigmoidLayer(L1, L2),
20           SoftmaxLayer(L2, 10)]
21 #####
22 # Training with mini-batch
23
24 history_loss = []
25 for i in range(100):
26     for X_batch, y_batch in batch_generator(X_train_norm, y_train_onehot, batch_size=
27         train_step(X_batch, y_batch, model=layers, lr=0.01)
28
29     y_pred = predict(X_train_norm, model=layers)
30
31     loss_ = loss(y_train_onehot, y_pred)
```

```
32 print(f"loss: {loss_}")
33 history_loss.append(loss_)
loss: 0.03925883897255526
loss: 0.038004272191363625
loss: 0.036786495095195965
loss: 0.035611016990751594
loss: 0.034476718193452015
loss: 0.03338130659343542
loss: 0.03232283177786184
loss: 0.03129994397442543
loss: 0.03031181264565108
loss: 0.02935764965903401
loss: 0.028436119295196398
loss: 0.027545426194667565
loss: 0.0266840979502417
loss: 0.025851516034161574
loss: 0.025047666605318382
loss: 0.02427272298155931
loss: 0.023526788402375712
loss: 0.02280952158869223
loss: 0.022119973458726617
loss: 0.021456830811321003
loss: 0.02081872064463875
loss: 0.020204374727415497
loss: 0.01961264255639054
loss: 0.019042431105358745
loss: 0.018492652419631467
loss: 0.017962229654416087
loss: 0.017450179522379105
loss: 0.016955735259437874
loss: 0.016478397193631717
loss: 0.0160177630573891
loss: 0.015573300279061269
loss: 0.015144474923905788
loss: 0.01473090041349397
loss: 0.014332272819654573
loss: 0.013948308841495909
loss: 0.013578643961046162
loss: 0.013222588923803383
loss: 0.01287886237925939
loss: 0.012545830462031277
loss: 0.012222426098866309
loss: 0.01190860419892295
loss: 0.011604687310828232
loss: 0.011310705017344134
loss: 0.01102631893475346
loss: 0.010750995096879011
loss: 0.010484168637127922
loss: 0.010225335195610362
loss: 0.009974082783077987
loss: 0.009730097161987077
loss: 0.009493154079245615
loss: 0.009263101562516366
loss: 0.009039838834588174
loss: 0.008823298778563402
loss: 0.008613435616265396
loss: 0.00841021450577303
loss: 0.008213598618868488
loss: 0.008023531858458827
loss: 0.007839919660444054
```

```
1 plt.plot(history_loss)
```

```
[<matplotlib.lines.Line2D at 0x7f1a690cba90>]
```



```
1 y_pred = predict(X_test_norm, model=layers)
2 y_pred = np.argmax(y_pred, axis=1)
3
4 print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.97	0.98	0.97	980
1	0.98	0.99	0.98	1135
2	0.95	0.96	0.96	1032
3	0.94	0.94	0.94	1010
4	0.94	0.96	0.95	982
5	0.92	0.93	0.93	892
6	0.96	0.95	0.96	958
7	0.96	0.96	0.96	1028
8	0.93	0.92	0.93	974
9	0.94	0.93	0.93	1009
accuracy			0.95	10000
macro avg	0.95	0.95	0.95	10000
weighted avg	0.95	0.95	0.95	10000

```
1 from tensorflow.keras.layers import Dense, Input
2 from tensorflow.keras.models import Model
3
4 inp = Input(shape=(784,))
5 x = Dense(units=64, activation="sigmoid")(inp)
6 x = Dense(units=64, activation="sigmoid")(x)
7 y = Dense(units=10, activation="softmax")(x)
8
9 model = Model(inputs=inp, outputs=y)
10 model.summary()
11
```

Model: "model"

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 784)]	0

dense (Dense)	(None, 64)	50240
dense_1 (Dense)	(None, 64)	4160
dense_2 (Dense)	(None, 10)	650

```

=====
Total params: 55,050
Trainable params: 55,050
Non-trainable params: 0
=====

```

```
1 model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=['acc'])
```

```
1 model.fit(X_train_norm, y_train_onehot, batch_size=512, epochs=50, validation_split
```

```

Epoch 1/50
94/94 [=====] - 2s 9ms/step - loss: 1.9660 - acc: 0.49
Epoch 2/50
94/94 [=====] - 1s 7ms/step - loss: 1.0634 - acc: 0.79
Epoch 3/50
94/94 [=====] - 1s 7ms/step - loss: 0.6131 - acc: 0.87
Epoch 4/50
94/94 [=====] - 1s 7ms/step - loss: 0.4340 - acc: 0.89
Epoch 5/50
94/94 [=====] - 1s 6ms/step - loss: 0.3514 - acc: 0.91
Epoch 6/50
94/94 [=====] - 1s 7ms/step - loss: 0.3047 - acc: 0.91
Epoch 7/50
94/94 [=====] - 1s 7ms/step - loss: 0.2728 - acc: 0.92
Epoch 8/50
94/94 [=====] - 1s 7ms/step - loss: 0.2487 - acc: 0.93
Epoch 9/50
94/94 [=====] - 1s 7ms/step - loss: 0.2287 - acc: 0.93
Epoch 10/50
94/94 [=====] - 1s 6ms/step - loss: 0.2120 - acc: 0.94
Epoch 11/50
94/94 [=====] - 1s 8ms/step - loss: 0.1963 - acc: 0.94
Epoch 12/50
94/94 [=====] - 1s 7ms/step - loss: 0.1837 - acc: 0.94
Epoch 13/50
94/94 [=====] - 1s 6ms/step - loss: 0.1717 - acc: 0.95
Epoch 14/50
94/94 [=====] - 1s 7ms/step - loss: 0.1606 - acc: 0.95
Epoch 15/50
94/94 [=====] - 1s 7ms/step - loss: 0.1506 - acc: 0.95
Epoch 16/50
94/94 [=====] - 1s 6ms/step - loss: 0.1417 - acc: 0.95
Epoch 17/50
94/94 [=====] - 1s 7ms/step - loss: 0.1335 - acc: 0.96
Epoch 18/50
94/94 [=====] - 1s 7ms/step - loss: 0.1260 - acc: 0.96
Epoch 19/50
94/94 [=====] - 1s 7ms/step - loss: 0.1189 - acc: 0.96
Epoch 20/50
94/94 [=====] - 1s 7ms/step - loss: 0.1126 - acc: 0.96
Epoch 21/50
94/94 [=====] - 1s 6ms/step - loss: 0.1068 - acc: 0.97
Epoch 22/50

```

```

94/94 [=====] - 1s 7ms/step - loss: 0.1013 - acc: 0.97
Epoch 23/50
94/94 [=====] - 1s 7ms/step - loss: 0.0961 - acc: 0.97
Epoch 24/50
94/94 [=====] - 1s 7ms/step - loss: 0.0915 - acc: 0.97
Epoch 25/50
94/94 [=====] - 1s 6ms/step - loss: 0.0870 - acc: 0.97
Epoch 26/50
94/94 [=====] - 1s 7ms/step - loss: 0.0828 - acc: 0.97
Epoch 27/50
94/94 [=====] - 1s 6ms/step - loss: 0.0788 - acc: 0.97
Epoch 28/50
94/94 [=====] - 1s 7ms/step - loss: 0.0750 - acc: 0.97

```

```

1 y_pred = model.predict(X_test_norm)
2 y_pred = np.argmax(y_pred, axis=1)
3
4 print(classification_report(y_test, y_pred))

```

	precision	recall	f1-score	support
0	0.98	0.99	0.98	980
1	0.99	0.99	0.99	1135
2	0.97	0.98	0.97	1032
3	0.96	0.98	0.97	1010
4	0.98	0.97	0.97	982
5	0.97	0.96	0.97	892
6	0.97	0.98	0.98	958
7	0.96	0.98	0.97	1028
8	0.97	0.96	0.97	974
9	0.98	0.95	0.96	1009
accuracy			0.97	10000
macro avg	0.97	0.97	0.97	10000
weighted avg	0.97	0.97	0.97	10000

